Tesla Rack Control Implementation Roadmap

From v4.2 bench validation through
RC integration to openpilot

Project	Tesla Rack Control
Vehicle	2013 Tesla Model S, pre-AP, post-May 2013 build
Current version	v4.2.0-dev
Branch	dev/v4.2-prnd
Authors	Derek Nagel · Jordan · Charlie Yonkura · Claude
Document date	2026-05-07

This document plans implementation only. Read PROJECT_MEMORY.md for the canonical reference of verified facts. Read SAFETY.md before any field test.

1. Executive summary

The tesla-rack-control project commands a 2013 Tesla Model S EPAS rack from a laptop over CAN, with the patched rack firmware accepting `0x488 DAS_steeringControl` directly. As of v4.2.0-dev (2026-05-07), the steering control loop is production-ready for on-jacks tests at ±60 degrees and is ready for in-motion validation at 3-5 mph in a parking lot.

The end goal is a fully RC-controlled car: laptop or Spektrum DX8 transmitter via Arduino commands steering, throttle, brake, and gear shift over CAN, with eventual openpilot integration once the control surfaces are stable.

Standstill steering with wheels on the ground is out of scope. Per Tesla's own production behavior and the physics of static tire friction, full-authority steering at literal zero speed is not achievable without invasive hardware modification. The realistic equivalent is a **rest-and-roll** primitive: brief brake release, tiny creep, steer, re-apply brake. This is what every modern park-assist system does.

Current state at a glance

State	Status	Where verified
Steering on jacks (±60°)	Production	session 232457
Steering in-motion (3-5 mph)	Untested	Phase 1
Standstill ground steering	Out of scope	physics
Gear shift command	Bench-correct	commit eae24c4
30 MPH MODE in-car	Refused (safe)	session 000935
Brake / gear / DI speed read	Production	session 002425
Theory C flicker	Resolved	Section 8
ESP contention	Resolved	Section 9
Throttle / brake / shift RC	Not started	Phase 2-4

2. System overview

The project converges two input paths onto the chassis CAN bus. The laptop path (currently in production) uses a SYS TEC USB-CAN adapter and the Python `tesla_control.py` program. The future Spektrum DX8 path will use an Arduino as an SBUS-to-CAN bridge. Only one path is active at a time -- they share the same CAN arbitration ID space.

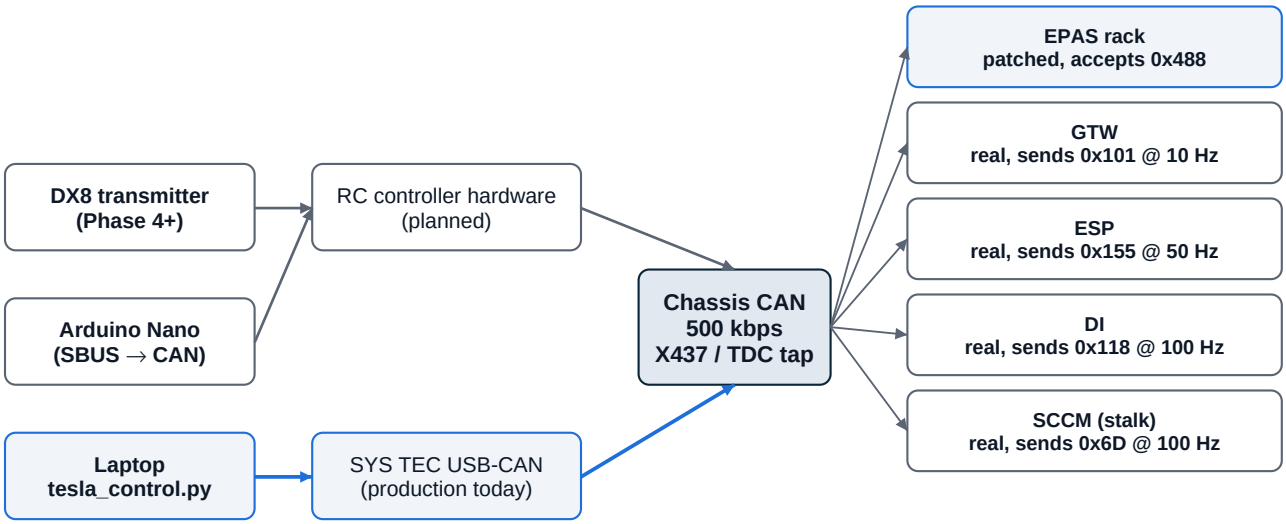
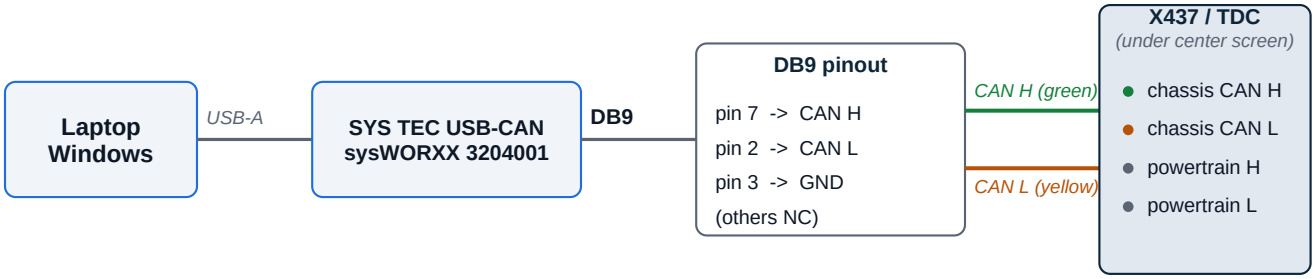


Figure 1. System block diagram. Two input paths converge on the chassis CAN bus; only one is active.

Hardware wiring

The SYS TEC USB-CAN adapter connects to the Tesla chassis CAN bus via the X437 / TDC connector under the center screen. This is the most reliable tap point on pre-AP cars; OBD-II pins 1 and 9 are sometimes populated with chassis CAN but vary by build date.



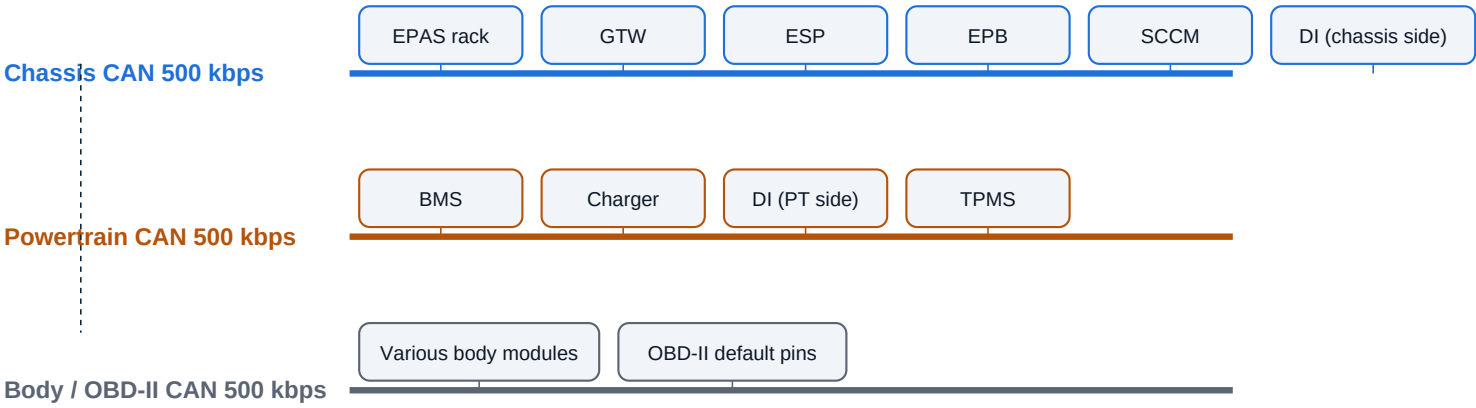
Verification before live tap:

1. Multimeter: ~2.5V on both CAN H and CAN L when bus is idle (recessive state).
2. Run can_sniffer.py at 500 kbps. Two or more known Tesla IDs visible confirms chassis CAN.
3. If pins 1/9 of OBD-II are populated on this car, that pair also reaches chassis CAN.

Figure 2. SYS TEC USB-CAN to X437 / TDC chassis CAN tap. Verified pin assignments.

3. CAN bus & protocol

Pre-AP Model S has three independent CAN buses. We use the chassis bus exclusively. Verified against the Tinkla wiki and TeslaMotorsClub reverse-engineering threads.



X437 / TDC under center screen exposes all 3 buses on different pin pairs.

Figure 3. Pre-AP Model S CAN bus topology. Three independent buses; we tap chassis at X437 / TDC.

Messages we use

All bit positions verified against the opendbc `tesla_can.dbc` file. Two integrity algorithms are in play: a simple sum checksum for the steering control set, and CRC-8/SAE-J1850 (poly `0x1D`) for the gear shift and stalk messages. Both algorithms verified against BogGyver/panda's reference implementation and 18,414 real captured stalk frames.

ID	Name	Direction	Rate	Notes
`0x488`	DAS_steeringControl	TX	50 Hz	Steering command (4 B, sum checksum)
`0x214`	EPB_epasControl	TX	10 Hz	Required keepalive (3 B, sum checksum)
`0x101`	GTW_epasControl	TX optional	20 Hz	Off when real GTW alive (3 B)
`0x155`	ESP_B fake speed	TX optional	200 Hz	30 MPH MODE only (8 B)
`0x6D`	SBW_RQ_SCCM	TX (burst)	100 Hz	Gear shift (4 B, CRC-8/J1850)
`0x370`	EPAS_sysStatus	RX	~100 Hz	Rack status (8 B)
`0x118`	DI_torque2	RX	~100 Hz	Gear, brake, DI speed (6 B)

Sign convention (verified by Jordan in-car 2026-05-03): positive commanded angle = wheel turns RIGHT.

4. Phased plan

Six phases from current state to long-term openpilot integration. Each phase has explicit acceptance criteria. The roadmap is gated: progression depends on the previous phase's outcome (see Figure 5).

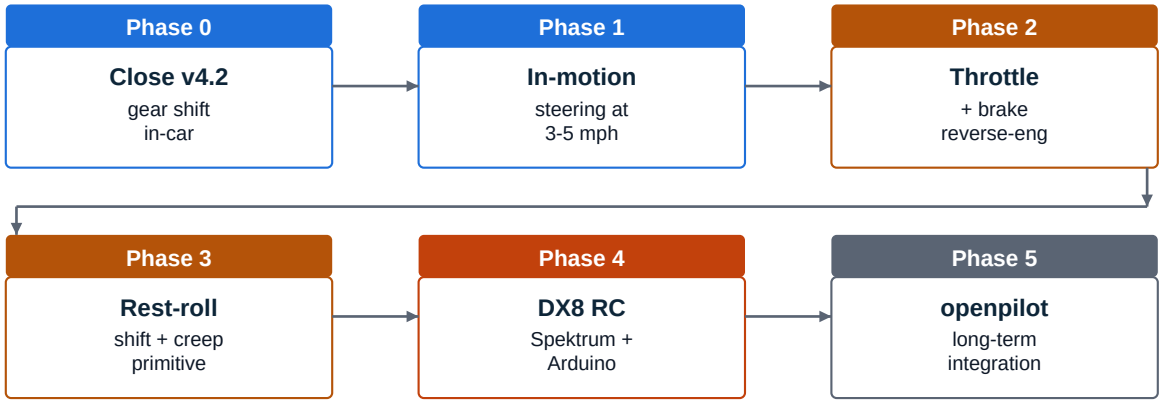


Figure 4. Six-phase plan. Color-coded by status: blue = active or imminent, yellow = upcoming, gray = long-term.

Phase 0 · Close out v4.2

Timing: this week

Validate the gear shift fix in-car. The byte 0/1 fix from commit `eae24c4` is bench-correct (matches all six captured shift frames byte-for-byte) but has not yet been re-tested in-car.

Acceptance: Clicking N/D/R in the GUI causes the Gear cell to update within 300 ms with brake pressed.

Phase 1 · In-motion steering

Timing: next week

Drive the car at 3-5 mph in a safe parking lot, command steering from the laptop. Real ESP reports actual speed, rack opens its at-speed envelope honestly, no spoofing required. This is the cleanest possible validation of the steering control loop.

Acceptance: Rack tracks commanded angles within 2 deg divergence at 3-5 mph, no E-STOPs, no flicker, session log saved to repo.

Phase 2 · Throttle & brake reverse engineering

Timing: weeks 2-4

Identify and command the CAN messages for go-pedal and brake-pedal on this 2013 Model S. Pre-AP uses vacuum-based braking; full software brake authority may not be achievable without hardware additions.

Acceptance: Laptop can command 0-10% throttle and 0-30% brake from a stationary rolled position, with auto-disengage on any anomaly.

Phase 3 · Rest-and-roll primitive

Timing: week 4

Combine steering + gear control such that the laptop holds the car in P, commands shift to D, briefly releases brake, executes a small steering command during the creep, re-applies brake, returns to P. Mirrors Tesla's own Autopark routine.

Acceptance: Scripted maneuver completes from stop, moves the car ~1 ft, returns to stop with brake applied.

Phase 4 · Spektrum DX8 + Arduino bridge

Timing: weeks 5-6

Replace the laptop GUI with a DX8 transmitter as primary control. Spektrum receiver on Arduino Nano UART, MCP2515 CAN module wired into chassis CAN. Channel mapping: left stick X = steering, right stick Y = throttle, switch = gear.

Acceptance: DX8 transmitter steers, accelerates, brakes, and shifts the car. Laptop is optional (used for logging only).

Phase 5 · openpilot integration

Timing: long-term

Out of scope for v4.x. Once steering + throttle + brake + shift are all CAN-commandable, openpilot's controlsd can emit those commands instead of the DX8. Likely path: fork BogGyver, integrate our improvements, write the safety model in the panda firmware.

Acceptance: openpilot drives the car with our control surfaces. Specific success criteria deferred until Phase 4 lessons land.

5. Decision flow

Each phase has an outcome that gates progression. If a phase fails its acceptance criteria, the project does not blindly continue -- we either iterate within the phase, scope-reduce, or pivot.

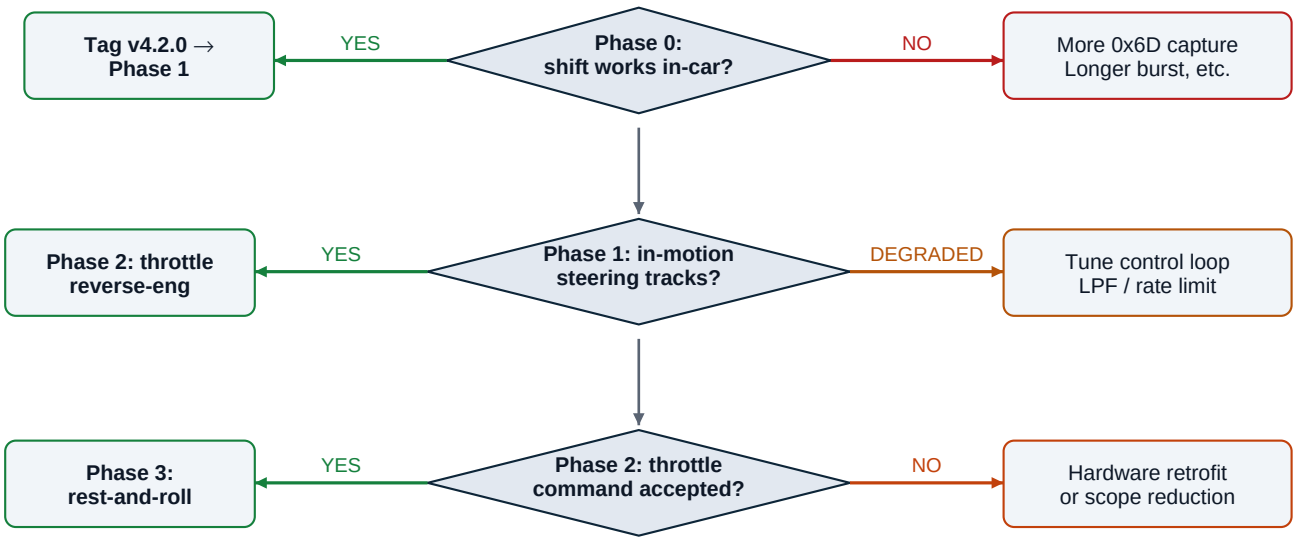


Figure 5. Decision tree after each phase. Diamonds are decision points; rectangles are the resulting next action.

6. Risks & mitigations

Risk	Probability	Impact	Mitigation
Rack stall torque insufficient even at full envelope	Medium	High	Rest-and-roll (Phase 3)
Pre-AP vacuum brake limits brake authority	Medium	Medium	Scope-limit Phase 2
Throttle reverse engineering takes longer than 3 wks	High	Medium	Pre-budget 4-5 weeks
Real stalk wins 0x6D arbitration after byte fix	Low	Low	Longer burst, faster rate
Repeated rack reflash bricks the rack	Low	Critical	Don't reflash without reason
ESP contention on second toggle attempt	Mitigated	--	Pre-flight check refuses (v4.2)
3X-style 0x488 contention on second device	Mitigated	--	Bus diag panel auto-flags (v4.2)

Anti-patterns to avoid: 30 MPH MODE on a live in-car bus (causes ESP cascade); leaving a comma 3X on chassis CAN while running tesla_control.py (Theory C contention); trusting the DBC blindly for CRC-protected commands (we caught two missing fixed-bit fields by capturing real stalk frames). All preventable; all already mitigated in v4.2.

7. Safety summary

The full safety model lives in `SAFETY.md` and `tesla_control.py`. This section is a one-page summary for quick reference.

Hard rules

• Front wheels off the ground or tie rods disconnected before any active testing.

• No passengers in or near the car during testing.

• No driving on a public road. Ever.

• Driver hands off the wheel during ENGAGE.

• 30 MPH MODE on jacks only -- never with the real ESP module on the bus.

Defense in depth (in tesla_control.py)

• Hard angle clamp (180 deg in software, ~60 deg enforced by rack at standstill).

• Rate limit (150 deg/sec) and 150 ms low-pass filter on user input.

• RX timeout watchdog: 0x370 lost > 500 ms triggers E-STOP.

• Bus error watchdog: > 50 CAN error frames triggers E-STOP.

• Loop overrun watchdog: 50 Hz TX loop late > 100 ms triggers E-STOP.

• EAC bounce watchdog: > 5 EAC transitions in 1 second triggers E-STOP.

• Real-motion auto-disengage if DI vehicle speed > 1 mph while engaged.

• Gear-out-of-park auto-disengage if gear leaves P while engaged.

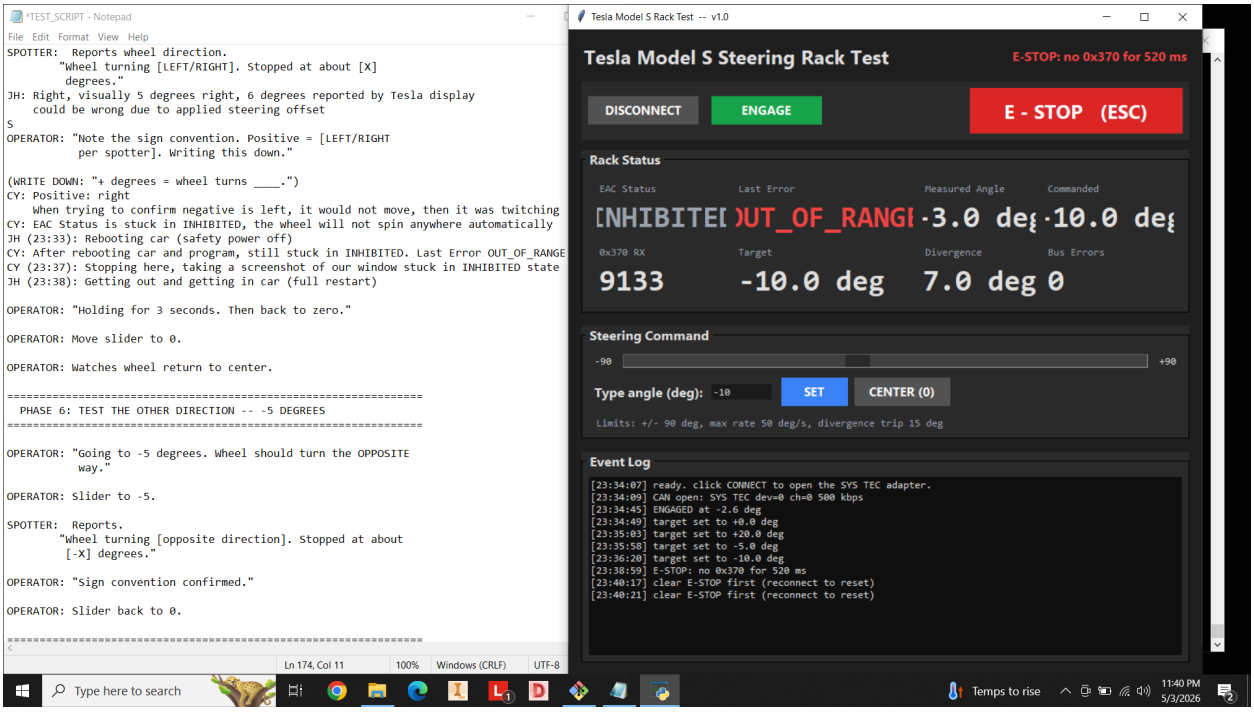
• Park-to-engage gate: ENGAGE refused if gear is not P (when DI visible).

• 30 MPH MODE pre-flight check: refuses if real ESP traffic detected.

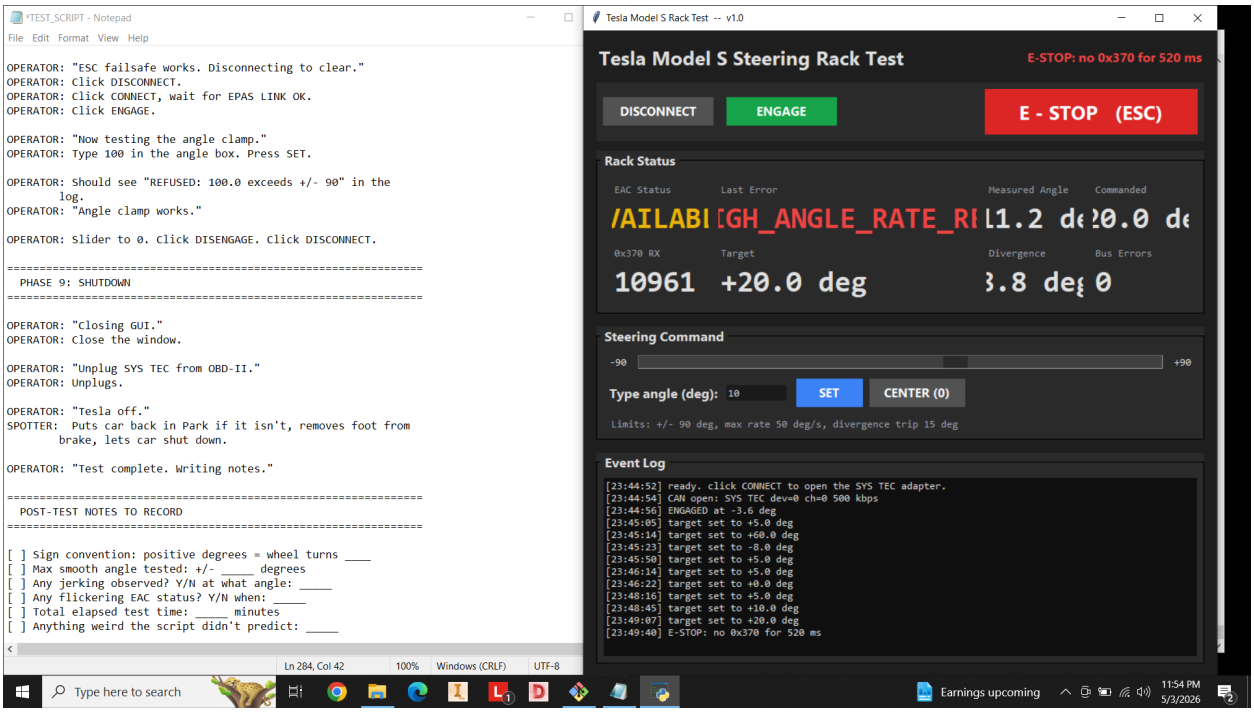
• Four manual E-STOP paths: red button, ESC, Q, window close.

Appendix A. Field test evidence

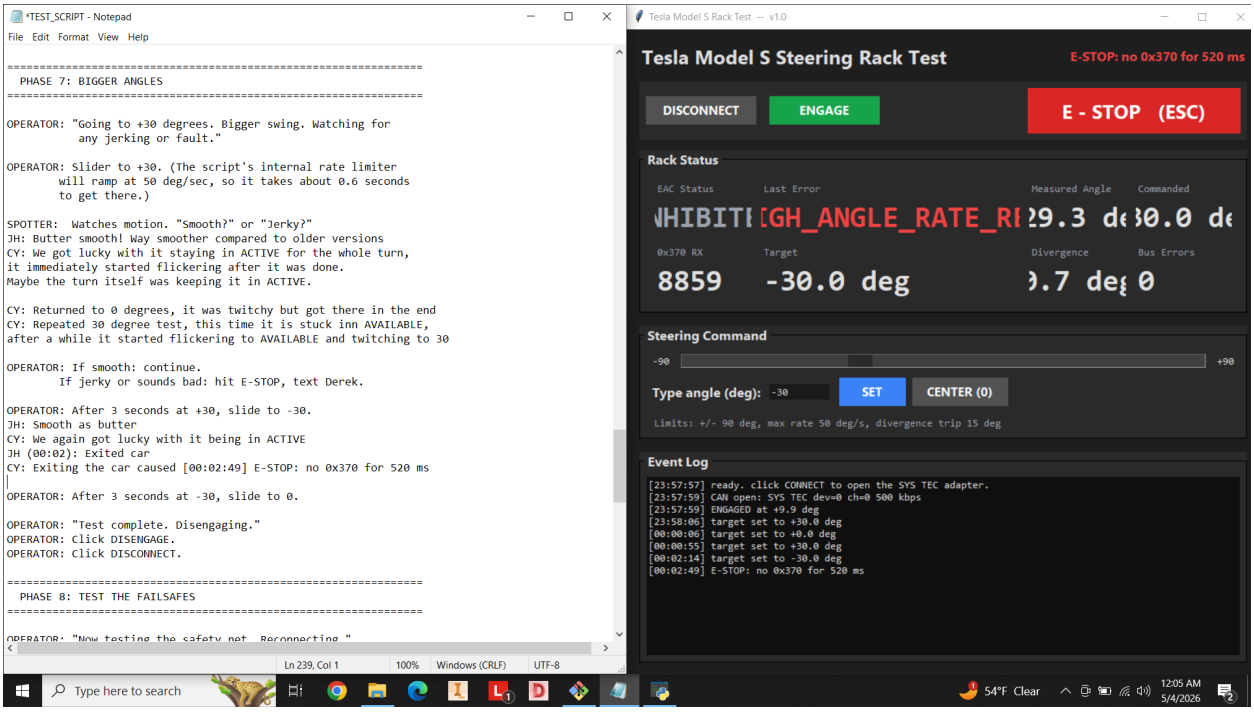
Screenshots from the May 2026 field test sessions, captured by Charlie. These show prior versions of the GUI under load (v2 / v3 architectures); the current v4.2 GUI is documented in `docs/GUIDE.md`.



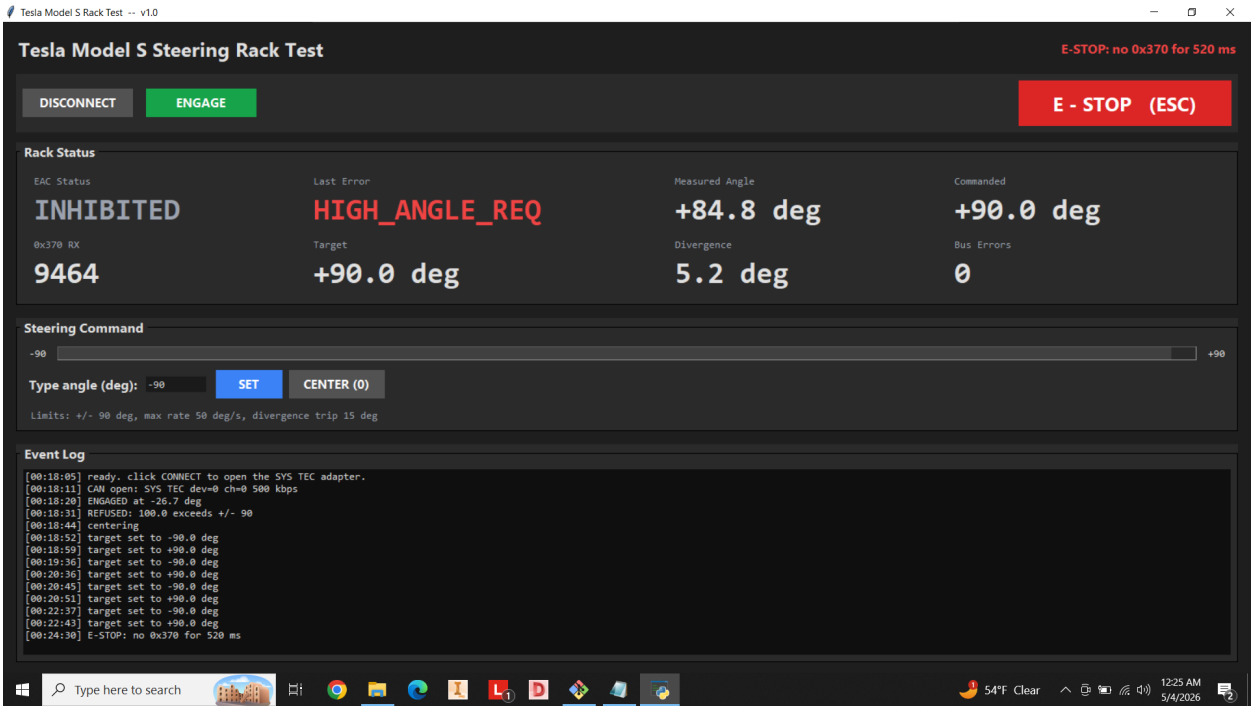
Session 03 May 2026 -- early test script + GUI



Session 03 May 2026 23:54 -- HIGH_ANGLE_RATE_REQ flicker visible



Session 04 May 2026 00:06 -- HIGH_ANGLE_RATE_REQ during active test



Session 04 May 2026 00:25 -- HIGH_ANGLE_REQ at +90° standstill

Appendix B. References

Verified sources used in this roadmap

gregjhogan/tesla-pre-ap-epas-patch

https://github.com/gregjhogan/tesla-pre-ap-epas-patch

EPAS firmware patch source. Three patched bytes verified at addresses 0x031750, 0x031892, 0x031974.

BogGyver/panda safety_tesla.h

https://github.com/BogGyver/panda/blob/tesla_unity_dev/board/safety/safety_tesla.h

Reference panda firmware. tesla_compute_crc lookup table verified to use polynomial 0x1D (CRC-8/J1850).

opendbc tesla_can.dbc

https://github.com/BYDcar/opendbc-byd/blob/master/tesla_can.dbc

Bit positions and enums for all Tesla CAN messages used.

Tinkla wiki

https://tinkla.us/index.php/Welcome_to_Tinkla!

Pre-AP retrofit ecosystem. CAN bus topology and OBD-II pin info.

Tesla recall: loss of EPAS at 0 mph

https://www.tesla.com/support/recall-vehicle-firmware-correct-loss-of-epas

Confirms Tesla's production EPAS is intended to assist at 0 mph.

comma openpilot tools/joystick

https://github.com/commaai/openpilot/tree/master/tools/joystick

Reference implementation of joystick-based CAN steering control.

MUXSAN 3-port CAN MITM bridge

https://www.tindie.com/products/muxsan/can-mitm-bridge-3-port-rev-25/

Commercial precedent for the hardware MITM bridge approach (Phase 2 alt).

Open-source CAN bridge by Emile Nijssen

https://www.hackster.io/news/emile-nijssen-s-open-source-can-bridge-makes-automotive-man-in-the-middle-a-cinch-b4dfeb952b04

Open-source MITM precedent for cheaper alternative.

Project-internal verified artifacts

`field_testing/captures/20260507_011220_shift_diagnostic/capture.csv` -- 18,414 real stalk frames captured by Charlie 2026-05-07. CRC verified, byte 0 / byte 1 fixed bits discovered. All shift positions confirmed.

`logs/session_20260506_232457.log` -- Theory C flicker fingerprint: 280+ EAC transitions in 67 seconds when 30 MPH MODE was toggled on.

`logs/session_20260507_000935.log` -- Pre-flight ESP check working: 30 MPH MODE refused twice with clear log message.